

Le Mans Université

Licence Informatique *2ème année*

Module 174UPO2 Rapport de Projet

MAZE-SHOOTER

DEQUIREZ Thomas

JAMMES Corentin

ASTIER Hugues

10 avril 2026

Table des matières

1	Introduction	2
2	Conception	2
2.1	Cahier des charges	2
2.2	Fonctionnalités du jeu	2
3	Organisation du projet	2
3.1	Repartition des tâches	2
3.2	Planning prévisionnel	2
3.3	Outils utilisés	2
4	Développement	3
4.1	Logique d'exécution du jeu	3
4.2	Création de la carte	4
4.3	Joueur, déplacement et gestion de la vie, coffre	4
4.4	Bots, leurs IA, spawn, boss	4
4.5	Armes, munitions, dégâts	5
4.6	Multijoueur, réseaux	6
5	Interface	8
6	Résultats et conclusion	8
6.1	Bilan temporel	8
6.2	Bilan de la conformité aux cahiers des charges	8
6.3	Améliorations possibles	8
6.4	Bibliographie	8
A	Annexes	8

1 Introduction

Ce rapport est une présentation du projet MAZE-SHOOTER réalisé par le groupe composé de Thomas, Corentin et Hugues. Il a plusieurs objectifs, en premier lieu, d'expliquer comment le projet s'est organisé, dans un second temps de rendre la lecture du code sur la page github (cf [Github]) accessible, et enfin, de présenter ses failles mais aussi ses forces.

2 Conception

2.1 Cahier des charges

Le cahier des charges à respecter suit les points suivants :

- Réaliser un jeu par groupe (de trois ici)
- Utiliser le langage C
- Utiliser une librairie graphique comme SDL ou RAYLIB
- Se servir de GIT
- Suivre un modèle de gestion de projet
- Utiliser doxygen

On a utilisé la librairie graphique Raylib, ainsi que GIT avec quatre branches (master, corentin, Thomas et HUGues) et on a suivi modèle linéaire.

2.2 Fonctionnalités du jeu

Dans MAZE-SHOOTER il y a deux modes de jeu principaux. Dans un premier temps, un mode solo, dans lequel le joueur doit survivre le plus longtemps possible face à des robots et des boss. Il doit en éliminer autant que possible. Et un mode multijoueur, dans lequel les joueurs s'affrontent entre eux dans un labyrinthe revisité pour faciliter les combats longue distance. Il est aussi possible de changer les textures via le menu "options".

3 Organisation du projet

3.1 Répartition des tâches

3.2 Planning prévisionnel

3.3 Outils utilisés

Pour ce projet, nous avons utilisé plusieurs outils pour faciliter le développement et la collaboration. Nous avons utilisé Git pour le contrôle de version, ce qui nous a permis de travailler simultanément sur différentes parties du code sans conflits majeurs. Pour la gestion des tâches et du planning, nous avons utilisé un diagramme de Gantt, où nous avons assigné des tâches à chaque membre de l'équipe. Pour la gestion du rendu graphique, nous

avons utilisé la librairie raylib, qui nous a permis de créer des graphismes 2D et 3D de manière efficace. Pour le rapport, nous avons utilisé overleaf, qui nous a permis de collaborer sur la rédaction du document en temps réel. Pour la documentation, nous avons utilisé Doxygen, qui nous a permis de générer automatiquement la documentation du code à l'aide de nos commentaires. Enfin, pour la communication, nous avons utilisé instagram, qui nous a permis de discuter en temps réel et de partager des idées rapidement.

4 Développement

4.1 Logique d'exécution du jeu

Dans cette section on explique la logique de fonctionnement du fichier main.c, et par la même occasion, la navigation du joueur dans le menu du jeu. Ce menu permet d'afficher différents boutons utiles à la navigation du joueur, cf figure A.1.

Pour comprendre comment fonctionnent la logique du jeu dans main.c, on a besoin d'une variable importante : `CurrentScreen`, c'est un pointeur sur un `Gamescreen`, qui décrit l'état de jeu actuel. Cette variable peut prendre diverses valeurs comme `NOUVELLE_PARTIE`, `CHARGER_PARTIE` ou encore `MENU`. Par défaut `CurrentScreen` vaut `MENU`.

Il faut maintenant comprendre comment le jeu tourne. Pour cela, dans main.c, on utilise une grande boucle while tournant tant qu'une variable `running` n'est pas mise à `FALSE` (ou que le joueur ne supprime pas sa fenêtre). Dans cette boucle, il faut distinguer la partie logique du jeu de la partie dessin, chaque partie correspond à un switch case. Chaque switch case regarde ce que vaut la variable `CurrentScreen`.

Ensuite, dans la partie logique on met à jours les variables globales, on appelle des fonctions « générique » qui initialisent ou mettent à jours nos variables globales (Player, Bot, etc). Un bon exemple est le suivant :

Exemple 1. `UpdateGame(...)` , cf [Github], cette fonction met à jours toute la partie logique du jeu en mode solo.

Une fois la logique à jours on peut dessiner. C'est le second switch qui entre en jeu et permet de lancer les bonnes fonctions de dessin toujours en fonction de la valeur de `CurrentScreen`. Désormais, pour comprendre menu.c, il suffirait lorsque `CurrentScreen` vaut `MENU` dans le premier switch qu'on appelle les fonctions gérant la souris et le clavier, puis dans le second on dessinent les boutons voulus.

En réalité par souci de simplicité, la partie logique et dessin est gérée simultanément, cependant pour la plus part des fichiers de ce projet les deux logiques sont bien distinctes.

4.2 Création de la carte

4.3 Joueur, déplacement et gestion de la vie, coffre

4.4 Bots, leurs IA, spawn, boss

Concernant les bots, ils sont guidés par une intelligence artificielle simple mais efficace. Ils empruntent le chemin le plus court vers le joueur dans le labyrinthe, mais dès qu'ils détectent le joueur à une certaine distance, ils commencent à le poursuivre en ligne droite ce qui permet d'éviter des calculs inutiles. Sinon si le joueur n'est pas visible ils utilisent un algorithme de recherche de chemin pour trouver le chemin le plus court vers le joueur, en évitant les obstacles du labyrinthe. Les bots ont également une capacité d'attaque, et lorsqu'ils sont suffisamment proches du joueur, ils peuvent lui infliger des dégâts. Les boss, quant à eux, sont des ennemis plus puissants avec des comportements plus complexes. Ils utilisent un sniper plutôt que le glock des bots normaux, ce qui fait qu'ils infligent un nombre de dégâts beaucoup plus élevé, leur IA est beaucoup plus précise (ils ne ratent pas une cible à moins qu'elle ne soit cachée derrière un mur ou qu'elle bouge très rapidement) et ils ont plus de points de vie que les bots normaux. Ils apparaissent à des moments clés du jeu pour offrir un défi supplémentaire au joueur. Néanmoins, ils se déplacent 2 fois moins vite que les bots normaux ce qui fait qu'ils sont plus faciles à fuir que les bots normaux, mais ils sont plus dangereux s'ils arrivent à vous toucher. Les bots et les boss ont la même logique de spawn, ils apparaissent à des endroits aléatoires du labyrinthe, bien que les boss apparaissent tous les 50 kills de bots classiques.

L'algorithme de recherche de chemin utilise 3 matrices. La première se nomme *visited* et contient des booléens indiquant si une case a déjà été visitée ou pas, la suivante s'appelle *parent* et contient quelle case a été visitée avant d'arriver à la case actuelle, et enfin la troisième matrice *queue* contient la liste des cases à visiter. Ces trois matrices permettent d'effectuer une recherche en largeur pour trouver le chemin le plus court vers le joueur. Donc comme si on versait de l'eau dans le labyrinthe à partir de la position du bot, et que l'eau se propageait dans toutes les directions en évitant les murs, la troisième matrice contiendrait la liste des cases où l'eau est présente à chaque instant, la première matrice indiquerait quelles cases ont déjà été touchées par l'eau, et la deuxième matrice indiquerait pour chaque case quelle est la case précédente d'où l'eau est venue. Lorsque l'eau atteint la position du joueur, on peut remonter le chemin en utilisant la deuxième matrice pour trouver le chemin le plus court vers le joueur.

Les bots et les boss sont du même type abstrait, ils sont tous les deux des *Entity*. Les *Entity* sont des tous les objets animés soit par un être humain comme par exemple les *ENTITY_PLAYER* qui ont été vu dans la partie sur les joueurs ou par une intelligence artificielle comme les *ENTITY_BOT* et les *ENTITY_BOSS*. Dans notre cas on va plutôt s'intéresser à ceux gérés par

des IA. Les bots et les boss ont des caractéristiques communes, comme une position générée aléatoirement parmi les cases du labyrinthe qui ne sont pas des murs, une direction gérée à de l'algorithme de recherche dans le labyrinthe. Ils ont également des comportements communs, comme se déplacer vers le joueur, attaquer le joueur lorsqu'ils sont suffisamment proches, et réagir aux dégâts qu'ils subissent. Cependant, les boss ont des caractéristiques plus puissantes que les bots normaux, comme plus de points de vie, une arme plus puissante, et une IA plus précise.

Les bots possèdent également une mécanique de salto. Il s'agit d'une mécanique qui est effectuée de manière aléatoire mais qui lui permet d'esquiver certains tirs adversaire. Lorsque le bot décide de faire un salto, il commence à tourner sur lui-même à une vitesse de 720 degrés par seconde pendant une durée de 0.5 secondes, ce qui lui permet d'esquiver les tirs ennemis pendant cette période. Cependant, le bot ne peut pas faire de salto indéfiniment, il doit attendre un certain temps avant de pouvoir en faire un autre. Le salto est une mécanique qui rend les bots plus difficiles à toucher et ajoute du dynamisme aux combats, mais elle est équilibrée par le fait que les bots ne peuvent pas en faire trop souvent.

En cas de mort d'un bot il réapparaît immédiatement à un autre endroit du labyrinthe, ce qui permet de maintenir une pression constante sur le joueur et d'assurer que le jeu reste dynamique. Cependant, les boss ne réapparaissent pas immédiatement après leur mort, ils réapparaissent si le score du joueur atteint un certain seuil, ce qui ajoute une dimension de progression au jeu et encourage le joueur à éliminer les bots pour faire réapparaître les boss.

4.5 Armes, munitions, dégâts

Cette section vise à expliquer comment les armes et leurs munitions sont gérées. On utilisera l'une des appellations suivante : projectile, munitions, la première sera davantage utilisée pour les collisions, la seconde pour la capacité du chargeur de l'arme.

Faisons un rapide tour d'horizon des fonctionnalités disponibles sur ce sujet, il existe quatre armes différentes :

1. un pistolet,
2. un sniper,
3. un fusil d'assault,
4. une grenade.

Chacune des armes dispose de ses propres caractéristiques (dégâts, capacité du chargeur, cadence de tir, ...). Dans le jeu, le joueur ne commence qu'avec la première arme, pour en obtenir d'autres, il doit satisfaire certaines conditions : augmenter son score en tuant d'autres bots, ou, récupérer des

coffres. Les paliers de score : 5,10 et 20 permettent de récupérer respectivement le sniper, le fusil et la grenade, cet ordre va de l'arme la moins à la plus forte. Mentionnons aussi qu'une texture par arme, et par projectile, est présente, voir section 5 « Interface », pour plus de détail.

Dans la partie logique du jeu (cf sous-section 4.1 « Logique d'exécution du jeu »), deux grandes fonctions interviennent : `ObtenirModeleArme(...)` et `ChangementArme(...)`.

La première récupère les caractéristiques de l'arme nécessaire pour tous les changements qu'opérera la seconde fonction. L'exemple suivant donne l'idée de ce qui se passe :

Exemple 2. Le joueur tape sur la touche F4, la fonction `ChangementArme(...)` vérifie que les conditions sont remplies, alors on récupère les caractéristiques de la grenade et les donne au joueur.

4.6 Multijoueur, réseaux

Dans le mode multijoueur, les joueurs s'affrontent dans un labyrinthe revisité pour faciliter les combats longue distance. Les joueurs peuvent se déplacer librement dans le labyrinthe et utiliser différentes armes pour éliminer leurs adversaires. Les armes disponibles ont été vu dans la section précédente sur les armes. L'objectif des joueurs dans ce mode est d'éliminer 25 fois son adversaire pour gagner la partie. Les joueurs peuvent également utiliser des power-ups qui apparaissent aléatoirement dans le labyrinthe pour obtenir des avantages temporaires, comme les caisses de heal réparties aléatoirement sur la map.

Le mode multijoueur utilise une architecture client-serveur, où un joueur agit en tant que serveur et l'autre joueur se connecte en tant que client. Le serveur est responsable de la gestion de l'état du jeu, de la synchronisation des actions des joueurs, et de la communication avec le client. Le client envoie ses actions au serveur, qui les traite et met à jour l'état du jeu en conséquence. La communication entre le serveur et les clients se fait via des sockets, ce qui permet une transmission efficace des données en temps réel.

Le réseau est géré de manière à assurer une synchronisation efficace entre les joueurs. Le serveur envoie régulièrement des mises à jour de l'état du jeu au client. La gestion du transfert des données, s'effectue via le protocole TCP dans la majorité des cas, qui garantit que les données sont transmises de manière fiable et dans le bon ordre. Le serveur écoute les connexions entrantes et gère les sessions de jeu pour le client. Ce protocole est couplé à une fonction qui permet de rendre les sockets non bloquants. Ce qui est d'une importance cruciale pour maintenir une expérience multijoueur fluide et réactive. En utilisant des sockets non bloquants, le serveur peut continuer à traiter les actions du jeu et à envoyer des mises à jour au client sans être

interrompu par des opérations de réseau, ce qui améliore considérablement la réactivité du jeu et l'expérience globale pour les joueurs.

Les joueurs ont deux méthodes pour se connecter à un serveur. La première méthode consiste à entrer l'adresse IP du serveur dans le menu de connexion du jeu, ce qui permet au client de se connecter directement au serveur néanmoins cette méthode requiert que le serveur ait au préalable ouvert le port 30000 et ait fait en sorte que lorsque l'on entre son adresse IP publique la demande de connexion a ce port redirige directement la requête vers sa machine. Dans ce cas c'est le protocole TCP qui est utilisé. La deuxième méthode consiste à jouer la partie en local, c'est à dire que les deux joueurs soient connectés à un même réseau. Dans ce cas le client peut se connecter automatiquement grâce à un broadcast local. Cette méthode est plus simple à mettre en place pour les joueurs, mais elle nécessite que les deux joueurs soient sur le même réseau, ce qui peut limiter les possibilités de jeu en ligne mais est néanmoins plus sûre pour le joueur qui héberge le serveur, car il n'a pas besoin d'ouvrir de port ou de gérer les redirections de port, ce qui peut être complexe et potentiellement risqué en termes de sécurité pour sa machine. Dans le cas du broadcast local, le protocole utilisé est UDP, qui permet une découverte rapide des serveurs disponibles sur le réseau local sans nécessiter de configuration préalable.

Pour la gestion du multijoueur, chacun gère ses données c'est à dire que si le client se fait tuer, il envoie un message au serveur pour lui indiquer qu'il a été tué, et le serveur met à jour son score. De même, lorsque le serveur est vaincu par son adversaire, il envoie un message au client pour lui indiquer qu'il a été tué, et le client met à jour son score. Cette approche est certes plus simple à mettre en place, mais elle peut être sujette à des problèmes de triches, car un joueur malveillant pourrait envoyer de faux messages pour manipuler le score. Cependant, pour un projet de cette envergure, cette méthode est suffisante pour assurer une expérience multijoueur fonctionnelle et agréable, tout en minimisant la complexité du développement.

5 Interface

6 Résultats et conclusion

6.1 Bilan temporel

6.2 Bilan de la conformité aux cahiers des charges

6.3 Améliorations possibles

6.4 Bibliographie

Références

[Github] Lien vers le projet GitHub : <https://github.com/thetsdz/MAZE-SHOOTER>

A Annexes

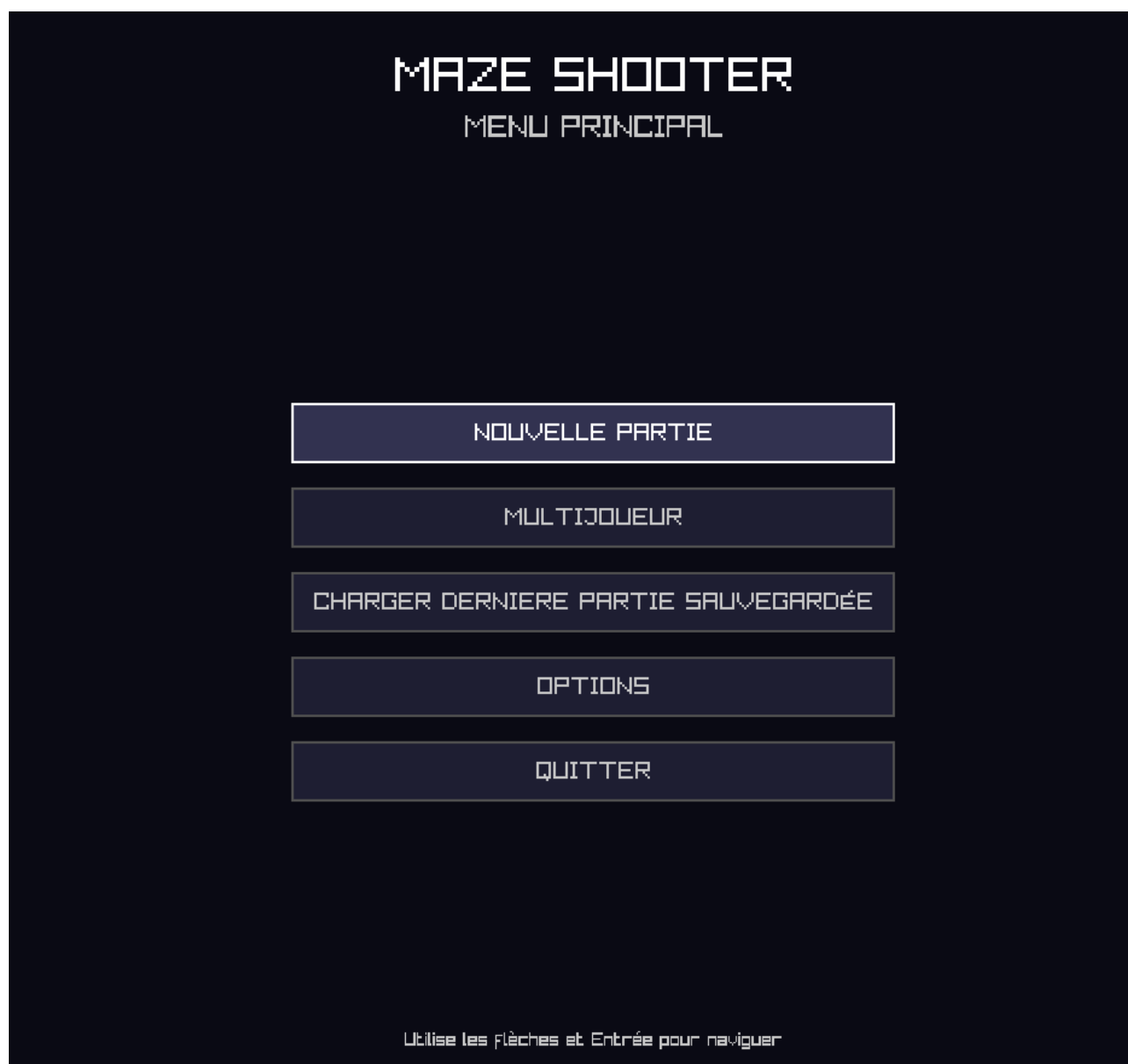


FIGURE A.1 – Capture d'écran du menu principal